Assignment 2

Questions

1. Why does BFS ignore the distances stored in the graph even though they are available to it?
   Breadth-First Search (BFS) explores graphs level by level based purely on the number of edges, designed to find the shallowest goal state. It treats all edge costs as equal and does not account for cumulative path weights.
2. Which two lines change when bfs is turned into dfs, and which of them makes DFS order dependent?
   The primary change is swapping the data structure from a FIFO Queue to a LIFO Stack. The Stack makes DFS order-dependent because the algorithm immediately branches down the most recently added child node
3. UCS tests for the goal when a state is popped from the heap. What could go wrong if it tested when the state was first generated?
   If Uniform-Cost Search (UCS) tests for the goal upon generation, it may return a suboptimal path.
4. Lab1reports *expanded* and *runtime_ms*. Which one should you use to compare BFS and UCS with a classmate's run on a different laptop, and why?
   Expanded would be more optimal as it depends only on the algorithm, graph, and tie-breaking while runtime_ms depends on CPU, load, Python version, and caching, so it isn't compatible across computers.
5. Which quantity does GBFS ignore that A⋆ uses, and which failure in Lab2 does that explain?
   GBFS ignores g, the cost already paid to reach a state, and ranks states only by h. A* uses f = g + h. Because GBFS ignores g, it can commit to a route that looks close to the goal but is more expensive overall. This explains a Lab 2 result where GBFS returns a path whose cost is higher than the optimal one.
6. Frontier states *X* and *Y* have *(g, h) = (8,2)* and *(3,6)*. Which does UCS expand first? GBFS? A⋆?
   UCS ranks by g: Y (3) beats X (8), so Y goes first. GBFS ranks by h: X (2) beats Y (6), so X goes first.
   A* ranks by f = g + h: X has f = 10 and Y has f = 9, so Y goes first.

7. Why must *h* be expressed in the same units as the edge costs?
   A* adds g and h to get f. If h is in different units, such as kilometres when edges are in minutes, the sum has no meaning. Also, h only guarantees optimal paths when it is a lower bound on the true cost in the same units. Otherwise, the ranking of states is wrong.
8. Lab2 computes $h\star$ by running Dijkstra on the reversed graph from the goal. Why reversed?
   h*(v) is the cheapest cost from v to the goal. Running Dijkstra from the goal on the reversed graph gives the cheapest cost from every node to the goal in a single run. Running it forward would need a separate search from each node.
9. Weighted A$\star$ with $w = 1.5$ changed the cost for KUL→KEF but not for SIN→EZE. What property of the SIN→EZE route explains this?
   If increasing the heuristic weight ($w = 1.5$) did not change the final route, it means the optimal path for SIN→EZE is heavily dominant. There were likely no competitive alternative routes that the inflated heuristic could force the algorithm to explore instead.

Create word file and answer all the above questions. Put this in your Git Hub repo and share with me your GitHub link for AI folder as well as this file link.

Sharing GitHub Repository link

Once you have pushed your Word file to GitHub, you can easily share the repository link with me.

Here is exactly how to find and share the correct link:

1. Copy the Repository URL

1. Go to GitHub and navigate to your repository page.
2. Look at your browser's address bar. The URL should look exactly like this:
   https://github.com
3. **Copy** that link.

2. Verify Your Repository Visibility (Crucial Step)

Before sending the link, make sure your TA can actually open it:

- **If it is a Public Repository:** Your TA will be able to see it immediately.
- **If it is a Private Repository:** Your TA will get a **404 Error (Page Not Found)** unless you explicitly invite them. So better to have public one.